



杭州电子科技大学

操作系统内核设计

Unfound OS

董洁 李嫣然 赵果利

2026 年 1 月

目录

第 1 章 摘要	1
1.1 Unfound 内核开发背景	1
第 2 章 系统概述	1
2.1 Unfound 整体架构概述	1
2.2 Unfound 代码结构	3
第 3 章 内存管理模块	3
3.1 内存管理模块框架概述	3
3.2 多分配器框架实现	4
3.3 分配器测试框架设计实现	6
第 4 章 文件系统模块	10
4.1 虚拟文件系统架构设计	10
4.2 具体实现	10
4.3 文件系统总结	17
第 5 章 总结与展望	17
5.1 项目总结	17
5.2 核心价值	18
5.3 现存局限	18
5.4 未来展望	18
第 6 章 致谢	19

第 1 章 摘要

1.1 Unfound 内核开发背景

本项目基于 ArceOS 开展开发工作。ArceOS 是一个采用 Rust 语言编写的实验性操作系统，最初以 Unikernel 架构实现，旨在通过内存安全与最小化攻击面提升系统可靠性。其核心设计以组件库 (library-based) 的形式，为操作系统内核提供基础设施与核心功能，所有模块均以独立的 Rust crate 组织，支持按需组合与灵活替换。

ArceOS 的架构高度模块化，涵盖以下关键组件：

- 硬件与资源抽象层：包括 axhal (硬件抽象层)、axalloc (安全内存分配器) 和 axdriver (统一驱动框架)，为跨平台支持奠定基础；
- 并发与任务管理：通过 axtask (轻量级任务调度) 和 axsync (异步同步原语)，原生支持高效异步与并发编程；
- 文件系统：基于 axfs 提供虚拟文件系统 (VFS) 抽象；
- 系统服务：如 axlog (日志系统)、arceos::posix_api (部分 POSIX 兼容接口)，便于移植传统用户态应用。

本项目基于 ArceOS 现有框架，旨在通过深度开发推动系统功能向完整操作系统演进。在初赛阶段，我们聚焦于两大核心模块——内存管理、文件系统，进行了系统性优化，显著提升了内核的稳定性与功能性，并大幅增强了对通用用户态程序的支持能力。

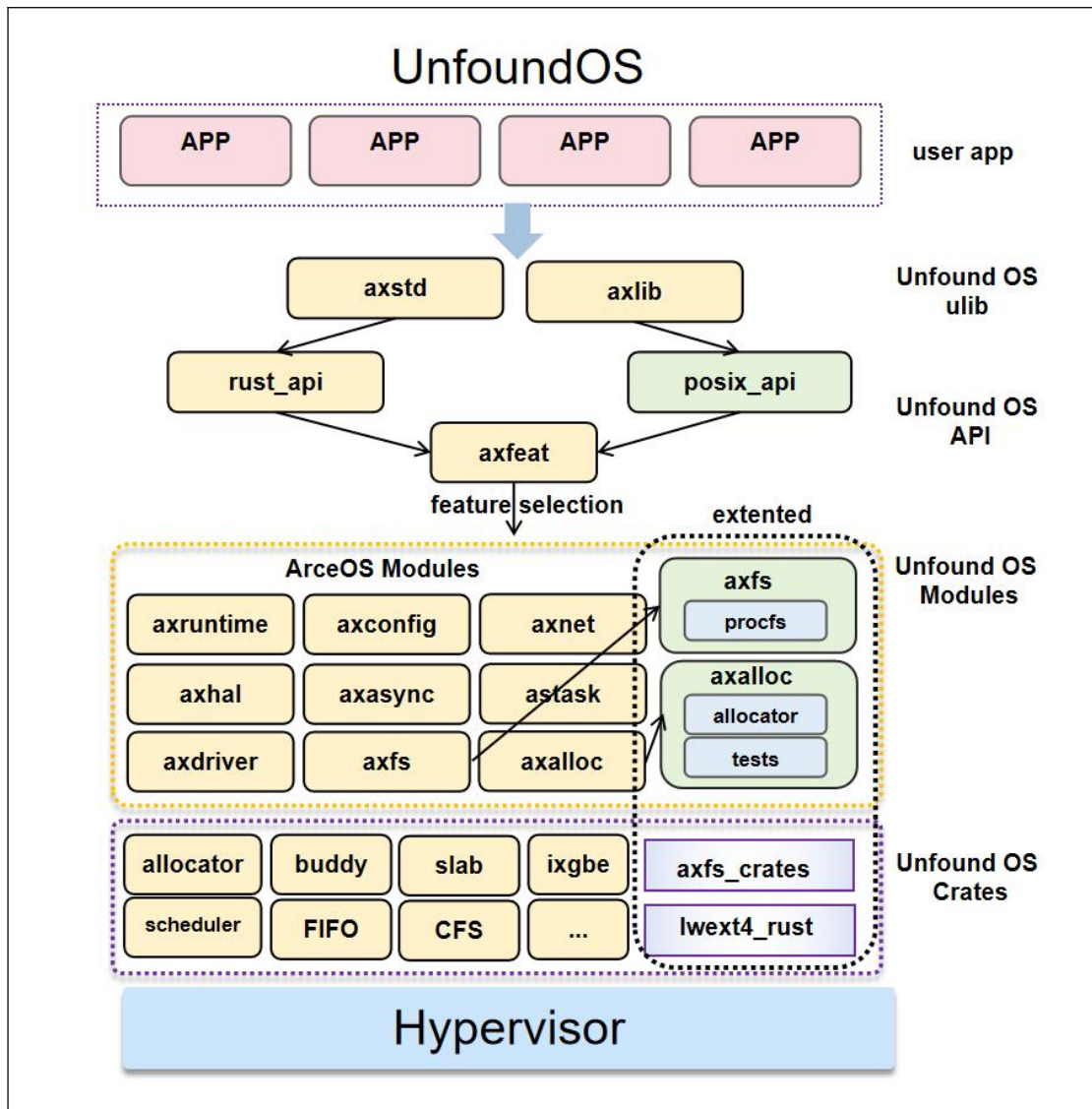
1.2 Unfound 整体内核设计

本项目的总体架构延续并深化了 ArceOS 的组件化设计理念，采用高度模块化的组织方式，将内核功能拆解为多个职责清晰、接口明确的独立模块，每个模块负责特定的功能。这样的设计使得内核更加灵活，易于扩展和维护，同时模块可以从内核中拆分出来，作为组件库提供给其他项目。

第 2 章 系统概述

2.1 Unfound 整体架构概述

内核的整体架构如下图所示：



在两大模块的创新主要体现在以下方面：

- 内存管理：在 UnfoundOS 不仅实现了 Buddy、Bitmap 和 Hybrid 三种物理内存分配算法，更打造了一套全维度测试框架，支持在用户态与内核态双环境下对吞吐量、碎片率、稳定性等指标进行量化对比；该模块深度集成于 ArceOS 的模块化架构，依托 Rust 的内存安全特性，既为内核提供高效可靠的物理页管理，又成为操作系统教学中连接理论与工程实践的实验平台。
- 文件系统：引入统一的 FileLike I/O 抽象模型，完整实现 POSIX *at 系列系统调用（如 openat、mkdirat、unlinkat），实现了任意路径动态挂载以及采用最长前缀匹配机制实现嵌套挂载，增强了文件系统的功能性和可扩展性。

两大模块协同演进，共同推动 UnfoundOS 从一个实验性 Unikernel 框架，成功转型为具备完整宏内核能力、可运行真实应用的操作系统原型。

2.2 Unfound 代码结构

- >modules: 内核组件 (内存、调度、驱动、网络、文件系统、日志等)
 - >axalloc: 内存分配器框架与测试模块
 - >axfs: 文件系统框架与测试
- >crates: crates 组件 (新增 axfs_crates、lwext4_rust)
- >api: 应用接口 (axfeat、arceos_api、arceos_posix_api)
- >ulib: 用户态库 (axstd、axlibc)
- >examples: 子系统示例与冒烟测试
- >configs: 平台配置
- >scripts、tools: 构建与 QEMU 运行脚本
- >doc: 设计文档与图表

第 3 章 内存管理模块

3.1 内存管理模块框架概述

UnfoundOS 的内存管理模块深度集成于 ArceOS 的模块化内核框架, 不仅覆盖通用内核开发的核心技能, 更充分体现了 ArceOS “以 crate 为单元、按需组合、安全优先” 的设计哲学。该模块不仅是一个功能完备的内核级物理内存分配器, 更是一个集算法验证、工程实践与系统思维培养于一体的实验教学平台。基于 Rust 语言的内存安全抽象, 模块实现了 Buddy (伙伴系统)、Bitmap (位图管理) 和 Hybrid (混合策略) 三种分配算法, 并配套一套全维度测试框架, 将抽象的内存管理理论转化为可观察、可对比、可复现的实验现象, 有效弥合理论教学与工程实践之间的鸿沟。

3.2 多分配器框架实现

多分配器框架的核心设计是通过 PageAllocator trait 抽象统一接口，以此支持 Buddy、Bitmap、Hybrid 等多种分配策略的无缝切换；该框架具备三大关键特性，即支持运行时动态选择分配器而无需重新编译内核、允许分配器的并行对比测试以便利性能评估与迁移验证、实现统一的内存池管理与生命周期控制。

核心代码：

```
01  /// Minimal allocator trait for page-level operations
02  used by the runtime
03  /// switching infrastructure.
04  pub trait PageAllocator: Send + Sync {
05      /// Return a short name of the allocator.
06      fn name(&self) -> &'static str;
07
08      /// Initialize allocator with a region starting at
09      `start_vaddr` of `size` bytes.
10      fn init(&self, start_vaddr: usize, size: usize) ->
11      Result<(), AllocError>;
12
13      /// Allocate contiguous pages: returns start
14      virtual address on success.
15      fn alloc_pages(&self, num_pages: usize,
16      align_pow2: usize) -> Result<usize, AllocError>;
17
18      /// Allocate contiguous pages at an exact start
19      address.
20      fn alloc_pages_at(
21          &self,
22          start: usize,
23          num_pages: usize,
24          align_pow2: usize,
25      ) -> Result<usize, AllocError>;
26
27      /// Deallocate contiguous pages starting from
28      `pos`.
29      fn dealloc_pages(&self, pos: usize, num_pages:
30      usize);
31
32      /// Optional: Return diagnostics (fragmentation and
33      free memory).
34      /// Default implementation returns (0.0, 0)
35      indicating no diagnostic data available.
```

```

        fn get_stats(&self) -> (f64, usize) {
            (0.0, 0)
        }
    }
}

```

具体实现:

modules/axalloc/src/allocators/buddy.rs - Buddy 分配器

modules/axalloc/src/allocators/bitmap.rs - Bitmap 分配器

modules/axalloc/src/allocators/hybrid.rs - Hybrid 混合分配器

运行时切换机制:

```

0  #[cfg(feature = "runtime-switch")]
1  pub mod runtime {
2      use super::PageAllocator;
3      use allocator::AllocError;
4      use core::option::Option;
5      use kspin::SpinNoIrq;
6
7      // Global storage for the runtime-selected page
8      allocator
9      static GLOBAL_PAGE_ALLOC: SpinNoIrq<Option<Box<dyn
10     PageAllocator>>> =
11         SpinNoIrq::new(None);
12
13     /// Try to set the global runtime allocator. Overwrites
14     any previous value.
15     pub fn set_runtime_allocator(a: Box<dyn
16     PageAllocator>) {
17         let mut slot = GLOBAL_PAGE_ALLOC.lock();
18         *slot = Some(a);
19     }
20
21     /// Helper to create an allocator by name.
22     pub fn make_by_name(name: &str) -> Result<Box<dyn
23     PageAllocator>, &'static str> {
24         match name {
25             "buddy" => {
26                 #[cfg(feature = "buddy")]
27                 { Ok(Box::new(crate::allocators::BuddyA
28                 llocator::new())) }
29                 #[cfg(not(feature = "buddy"))]
30                 { Err("buddy feature not enabled") }
31             }
32         }
33     }
34 }

```

```

1         "bitmap" => {
7             #[cfg(feature = "bitmap")]
1             { Ok(Box::new(crate::allocators::Bitmap
8 Allocator::new())) }
1             #[cfg(not(feature = "bitmap"))]
9             { Err("bitmap feature not enabled") }
2         }
0         "hybrid" => {
2             #[cfg(feature = "hybrid")]
1             { Ok(Box::new(crate::allocators::Hybrid
2 Allocator::new())) }
2             #[cfg(not(feature = "hybrid"))]
2             { Err("hybrid feature not enabled") }
3         }
2         _ => Err("unknown allocator name"),
4     }
2 }
8 }

```

3.3 分配器测试框架设计实现

UnfoundOS 分配器测试框架具备模块化、可扩展的测试能力，专门用于评估内存分配器的性能、碎片情况、稳定性及时间维度表现，其核心支持涵盖多方面测试需求，包括基础性能测试（分配成功率、分配与释放耗时、吞吐量）、碎片测试（外部与内部碎片率、元数据开销）、稳定性测试（分配失败率、内存泄漏检测、多线程竞争开销）、时间维度测试（1 小时、6 小时、24 小时的长期运行模拟）、运行时切换测试（分配器切换耗时与兼容性），以及内核兼容测试（适配嵌入式与内核环境的 no_std 测试能力）。

基础测试:

```

01 /// Run basic performance tests on an allocator.
02 ///
03 /// This measures:
04 /// - Allocation success rate
05 /// - Average allocation/deallocation time
06 /// - Throughput (ops/sec)
07 /// - Time distribution (min, max, p50, p99)
07 pub fn run_basic_performance_test<A: PageAllocator
08 + ?Sized>(
09     allocator: &A,

```

```

10     config: &BasicPerfConfig,
11 ) -> BasicPerformanceMetrics {
12     let mut metrics = BasicPerformanceMetrics::new();
13     let mut alloc_times: Vec<u64> = Vec::new();
14     let mut dealloc_times: Vec<u64> = Vec::new();
15     let mut allocated: Vec<(usize, usize)> = Vec::new();
16     // (addr, size_pages)
17
18     let mut rng = SimpleRng::new(42);
19     let test_start = Instant::now();
20
21     for _i in 0..config.num_allocations {
22         if !allocated.is_empty() && rng.next_f64() <
23 config.dealloc_during_alloc_ratio {
24             let idx = rng.next_usize(allocated.len());
25             let (addr, size) =
26 allocated.swap_remove(idx);
27             let start = Instant::now();
28             allocator.dealloc_pages(addr, size);
29             let elapsed = start.elapsed().as_nanos() as
30 u64;
31             dealloc_times.push(elapsed);
32         }
33
34         let size_idx =
35 rng.next_usize(config.allocation_sizes.len());
36         let size = config.allocation_sizes[size_idx];
37
38         let start = Instant::now();
39         let result = allocator.alloc_pages(size,
40 config.alignment);
41         let elapsed = start.elapsed().as_nanos() as
42 u64;
43
44         metrics.total_allocations += 1;
45
46         match result {
47             Ok(addr) => {
48                 metrics.successful_allocations += 1;
49                 allocated.push((addr, size));
50                 alloc_times.push(elapsed);
51             }
52             Err(_) => {
53                 metrics.failed_allocations += 1;

```

```

    }
}

metrics.calculate_success_rate();
metrics
}

```

碎片测试:

```

01 /// Collect a fragmentation snapshot from allocator
02 stats.
03 fn collect_fragmentation_snapshot<A: PageAllocator
04 + ?Sized>(
05     allocator: &A,
06     time_hours: f64,
07     total_memory: usize,
08     tracked_requested: usize,
09     tracked_actual: usize,
10 ) -> FragmentationSnapshot {
11     let (frag_rate, total_free) =
12 allocator.get_stats();
13     let max_contiguous = if total_free > 0 {
14         ((1.0 - frag_rate) * total_free as f64) as usize
15     } else {
16         0
17     };
18     let actual_allocated = if total_memory > total_free
19 {
20         total_memory - total_free
21     } else {
22         0
23     };
24     let internal_waste = if tracked_actual >
25 tracked_requested {
26         tracked_actual - tracked_requested
27     } else {
28         0
29     };
30     let mut snapshot = FragmentationSnapshot {
31         time_hours,
32         total_free_bytes: total_free,
33         max_contiguous_free_bytes: max_contiguous,
34         external_fragmentation_rate: frag_rate *

```

```

35 100.0,
36     total_allocated_bytes: actual_allocated,
37     internal_waste_bytes: internal_waste,
38     internal_fragmentation_rate: 0.0,
    };
    if tracked_actual > 0 {
        snapshot.internal_fragmentation_rate =
            (internal_waste as f64 / tracked_actual as f64) * 100.0;
    }
    snapshot
}

```

时间维度:

```

01 /// Run fragmentation tests over simulated time periods.
02 pub fn run_fragmentation_test<A: PageAllocator
03 + ?Sized>(
04     allocator: &A,
05     config: &FragmentationTestConfig,
06 ) -> FragmentationMetrics {
07     let mut metrics = FragmentationMetrics::new();
08     let mut rng = SimpleRng::new(42);
09     let test_start = Instant::now();
10     for _ in 0..config.num_iterations {
11         let size_idx =
12             rng.next_usize(config.allocation_sizes.len());
13         let size = config.allocation_sizes[size_idx];
14         if let Ok(addr) = allocator.alloc_pages(size,
15             config.alignment) {
16             allocator.dealloc_pages(addr, size);
17         }
18     }
19     metrics.total_time =
20         test_start.elapsed().as_secs_f64();
21     metrics
22 }

```

第 4 章 文件系统模块

4.1 虚拟文件系统架构设计

虚拟文件系统 (Virtual File System, VFS) 是现代操作系统中用于抽象和统一多种具体文件系统的核心机制。其核心思想是：将所有 I/O 请求首先路由至一个中间抽象层 (即 VFS)，由该层解析请求语义，并将其转发给底层具体的文件系统实现。这种设计使得上层应用无需关心底层存储格式，从而实现“一次编写，多后端运行”的目标。ArceOS 的虚拟文件系统 (VFS) 作为 I/O 子系统的基石，虽已提供基本的文件读写能力，并遵循统一抽象、灵活扩展与透明访问的设计原则，但其功能仍存在明显局限：缺乏对目录操作 (如创建、遍历和删除) 的完整支持，未实现 POSIX 标准中的关键 `*at` 系列系统调用 (如 `openat`、`mkdirat`、`unlinkat`)。

针对 ArceOS 虚拟文件系统 (VFS) 在目录操作、POSIX 兼容性及扩展能力等方面的不足，Unfound 提出了系统性改进，实现了以下关键创新：

- 统一 I/O 抽象模型
- 完整 POSIX 目录支持
- 动态多后端挂载机制：支持在任意路径 (如 `/mnt/usb`) 运行时挂载 FAT、EXT4 等文件系统，通过最长前缀匹配实现智能请求路由；

4.2 具体实现

1. 统一 I/O 抽象模型：与 ArceOS 一脉相承，Unfound OS 同样以 Rust 的 `trait` 机制为核心，对虚拟文件系统 (VFS) 进行了结构化抽象。我们在原有基础上进一步扩展了两套关键接口，分别对应文件系统级和节点级操作。具体定义如下：

```
01  /// VFS 文件系统级操作接口（对应整个文件系统实例）
02
03  /// 要求实现线程安全 (Send + Sync)，支持并发访问
04
05  pub trait VfsOps: Send + Sync {
06
07      /// 在指定路径挂载一个 VFS 节点（如将 FAT 分区挂载到
08      /mnt/usb)
09
10      fn mount(&self, path: &str, mount_point:
11      VfsNodeRef) -> VfsResult;
12
13
14      /// 卸载当前文件系统
```

```

15     fn umount(&self) -> VfsResult;
16
17     /// 格式化底层存储设备（如清空并初始化文件系统元数据）
18
19     fn format(&self) -> VfsResult;
20
21     /// 获取文件系统的统计信息（如总空间、可用空间等），用于
22
23     statfs 系统调用
24
25     fn statfs(&self, path: *const core::ffi::c_char,
26     info: *mut FileSystemInfo) -> VfsResult<usize>;
27
28     /// 返回该文件系统的根目录节点引用
29
30     fn root_dir(&self) -> VfsNodeRef;
31 }
32
33 /// VFS 节点级操作接口（对应单个文件、目录或设备节点）
34
35 /// 要求实现线程安全（Send + Sync）
36
37 pub trait VfsNodeOps: Send + Sync {
38
39     /// 打开节点时的回调（通常用于设备或特殊文件，普通文件可留
40
41     空）
42
43     fn open(&self) -> VfsResult { Ok(()) }
44
45     /// 关闭或释放节点资源时的回调
46
47     fn release(&self) -> VfsResult { Ok(()) }
48
49     /// 获取节点的基本属性（类型、大小、权限、时间戳等）
50
51     fn get_attr(&self) -> VfsResult<VfsNodeAttr>;
52
53     /// 获取节点的扩展属性（支持更精细的元数据，如纳秒级时间戳）
54
55     fn get_attr_x(&self) -> VfsResult<VfsNodeAttrX>;
56
57     /// 从指定偏移量读取数据到缓冲区（用于普通文件）
58
59     fn read_at(&self, offset: u64, buf: &mut [u8]) ->

```

```

VfsResult<usize>;

    /// 从指定偏移量写入数据（用于普通文件）
    fn write_at(&self, offset: u64, buf: &[u8]) ->
VfsResult<usize>;

    /// 读取目录项列表（用于目录节点，支持 ls、getdents 等）
    fn read_dir(&self, start_idx: usize, dirents: &mut
[VfsDirEntry]) -> VfsResult<usize>;

    /// 在当前目录下查找子路径对应的节点（用于路径解析，如
lookup("foo/bar"))

    /// 使用 Arc<Self> 以支持目录遍历时的引用传递
    fn lookup(self: core::sync::Arc<Self>, path: &str)
-> VfsResult<VfsNodeRef>;
}

```

我们在 VfsNodeOps 中新增了 get_attr_x、set_atime、set_mtime、xattr（扩展属性）等接口，支持更丰富的元数据管理，便于兼容 POSIX/EXT4 等高级特性。

2. 完整 POSIX 目录支持:

Unfound OS 在 root.rs 中实现了符合 POSIX 语义的完整目录操作能力，涵盖路径解析与核心目录管理功能，包括：

- 路径解析：支持相对路径、绝对路径及../.的规范化处理；
 - 节点生命周期管理：提供 create（创建文件/目录）、lookup（查找节点）、remove（删除节点）、rename（重命名）等关键操作；
 - 当前工作目录（CWD）支持：为用户态程序提供基于进程的当前目录上下文。
- 相关目录操作函数，具体定义如下：

```

01  /// 在指定目录（或当前工作目录）下查找给定路径对应的 VFS 节点。
02  ///
03  ///
04  /// - `dir`：可选的起始目录节点；若为 None，则从根目录开始解
05  析。
06  /// - `path`：要查找的相对或绝对路径（如 "a/b/c" 或
07  08  "/mnt/usb/file.txt"）。
09  10
11

```

```

12  ///
13  /// 返回查找到的节点引用。若路径以 '/' 结尾但目标不是目录，则
14  返回 `NotADirectory` 错误。
15
16  pub(crate) fn lookup(dir: Option<&VfsNodeRef>, path:
17  &str) -> AxResult<VfsNodeRef> {
18      if path.is_empty() {
19          return ax_err!(NotFound); // 空路径视为未找到
20      }
21      // 通过 parent_node_of 获取路径的父目录节点，然后在其下
22      lookup 最后一级组件
23      let node = parent_node_of(dir,
24      path).lookup(path)?;
25      // 如果路径以 '/' 结尾，但查找到的节点不是目录，则报错
26      if path.ends_with('/')
27      && !node.get_attr()?.is_dir() {
28          ax_err!(NotADirectory)
29      } else {
30          Ok(node)
31      }
32  }
33
34  /// 创建一个新目录。
35  ///
36  /// - `dir`: 可选的起始目录；若为 None，则从根目录开始解析路
37  径。
38
39  /// - `path`: 要创建的目录路径（如 "a/b/newdir"）。
40  ///
41  pub(crate) fn create_dir(dir: Option<&VfsNodeRef>,
42  path: &str) -> AxResult {
43      match lookup(dir, path) {
44          Ok(_) => ax_err!(AlreadyExists), // 目录已存在
45          Err(AxError::NotFound) => {

```

```

        // 父目录存在但最后一级不存在，尝试创建
        parent_node_of(dir, path).create(path,
VfsNodeType::Dir)
    },
    Err(e) => Err(e), // 其他错误（如权限、非目录中间
路径等）直接透传
}
}

/// 删除一个空目录。
///
/// - `dir`：可选的起始目录；
/// - `path`：要删除的目录路径。
///
pub(crate) fn remove_dir(dir: Option<&VfsNodeRef>,
path: &str) -> AxResult {
    // validation + permission checks then delegate to
parent.remove(path)
    // ...existing code...
}

```

POSIX 系统调用绑定与上层映射

上述 VFS 能力通过 `arceos_posix_api` 模块向上暴露为标准系统调用接口。具体实现位于：

```

1 /home/hen/unfound/api/arceos_posix_api/src/imp/fs_compl
  ex_backup.rs

```

该文件负责将底层 VFS 操作（如 `lookup`、`create_dir`、`remove_dir` 等）映射到对应的 POSIX 系统调用（如 `openat`、`mkdirat`、`unlinkat`、`renameat` 等），并处理路径解析、权限检查、错误码转换等细节，从而确保 GNU coreutils 等传统用户态程序能够正常运行。

3. 动态多后端挂载机制：

Unfound OS 的动态挂载能力在 `root.rs` 中实现，核心包括：

- > 挂载表 (Mount Table)：以线程安全的方式维护所有已挂载文件系统的元信息；
- > 挂载/卸载操作：支持在任意绝对路径（如 `/mnt/usb`）挂载新的文件系统，并完成目录节点创建与后端初始化；
- > 最长前缀匹配 (Longest Prefix Match)：在路径解析时，自动选择最具体的挂载点，确保嵌套挂载（如 `/mnt` 与 `/mnt/usb` 共存）的正确路由。

示 例 （`MountPoint` / `RootDirectory.mount` / `lookup_mounted_fs` / `find_mountpoint_and_fs`）具体定义如下：

```

01 /// 表示一个挂载点：包含挂载路径和对应的文件系统实例
02
03 struct MountPoint {
04     path: &'static str, // 挂载路径
05     (如 "/mnt/usb") , 生命周期为 'static 以简化管理
06
07     fs: alloc::sync::Arc<dyn VfsOps>, // 指向具体
08     文件系统的 Arc 引用，支持动态分发
09 }
10
11
12
13
14 /// 根目录管理器，负责统一调度主文件系统与所有挂载点
15
16 pub struct RootDirectory {
17     main_fs: alloc::sync::Arc<dyn VfsOps>, // 主文件系
18     统（通常挂载在 "/"）
19
20     mounts: spin::RwLock<Vec<MountPoint>>, // 所有挂载
21     点列表，使用读写锁支持并发访问
22 }
23
24
25
26
27 impl RootDirectory {
28     /// 在指定路径挂载一个新的文件系统。
29     ///
30     /// - `path`：挂载点路径（必须是绝对路径，且不能为根 "/"）；
31     /// - `fs`：要挂载的文件系统实例。
32     ///
33     pub fn mount(&self, path: &'static str, fs:
34     alloc::sync::Arc<dyn VfsOps>) -> AxResult {
35         if path == "/" {
36             return ax_err!(InvalidInput, "cannot mount
37             root filesystem");
38         }
39
40         // 在主文件系统中创建挂载点目录（如 /mnt/usb）
41         self.main_fs.root_dir().create(path,

```

```

45 FileType::Dir)?;
46         // 通知新文件系统：你已被挂载到该路径
47         fs.mount(path,
48 self.main_fs.root_dir().lookup(path)?);
49
50         // 注册挂载点
51         self.mounts.write().push(MountPoint { path,
52 fs });
53         Ok(())
54     }
55
56
57     /// 在所有挂载点中执行最长前缀匹配，并对匹配到的文件系统调用回调函数。
58
59     ///
60     /// - `path`：要查找的完整路径（如
61
62     "/mnt/usb/file.txt"）；
63
64     /// - `f`：回调函数，接收匹配到的文件系统和剩余子路径（如
65 fs 和 "file.txt"）。
66
67     ///
68     /// 用途：用于 lookup、open、stat 等操作的路由分发。
69     fn lookup_mounted_fs<F, T>(&self, path: &str, f: F)
70 -> AxResult<T>
71     where
72         F: FnOnce(alloc::sync::Arc<dyn VfsOps>, &str)
73 -> AxResult<T>
74     {
75
76         // 对 self.mounts.read() 进行最长前缀匹配（例如
77
78     "/mnt/usb" 比 "/mnt" 更优）
79
80         // ...existing code...
81     }
82
83     /// 查找给定路径所属的挂载点及其文件系统。
84
85     ///

```

```

    /// - `path`: 输入路径 (如 "/mnt/usb/data") ;

    /// - 返回: 匹配的挂载路径 (如 "/mnt/usb") 和对应的文件
    系统引用。

    ///

    /// 内部会先对路径进行规范化 (如处理 ".."、"."、多余 '/') ,
    再遍历挂载点列表寻找最佳匹配。

    pub fn find_mountpoint_and_fs(&self, path: &str) ->
    AxxResult<(alloc::string::String, alloc::sync::Arc<dyn
    VfsOps>)> {
        /// canonicalize + iterate mounts to find best
        match, return (mount_path, fs)
        /// ...existing code...
    }
}

```

4.3 文件系统总结

Unfound OS 在 ArceOS 模块化内核的基础上, 扩展虚拟文件系统 (VFS) 架构。通过统一 I/O 抽象模型 (FileLike trait), 将文件、目录及未来 I/O 对象纳入一致接口, 奠定“一切皆可描述符”的基础; 通过完整的 POSIX 目录支持, 实现 `openat`、`mkdirat`、`unlinkat`、`getdents` 等关键系统调用; 并通过动态多后端挂载机制, 支持在任意路径 (如 `/mnt/usb`) 运行时挂载 FAT、EXT4、RAMFS 等多种文件系统, 借助最长前缀匹配实现智能请求路由, 兼顾灵活性与 POSIX 兼容性。

第 5 章总结与展望

5.1 项目总结

Unfound OS 基于 ArceOS 模块化内核框架, 以 Rust 语言的内存安全特性为基石, 完成了从实验性 Unikernel 到具备完整宏内核核心能力的操作系统原型演进。项目聚焦内存管理与文件系统两大核心模块, 通过系统性设计与优化, 显著提升了内核的功能性、稳定性与兼容性, 为后续扩展奠定了坚实基础。

在内存管理模块, 项目创新构建了多分配器框架与全维度测试体系: 通过 `PageAllocator trait` 抽象统一接口, 实现了 `Buddy`、`Bitmap`、`Hybrid` 三种分配算法

的无缝集成与运行时动态切换，支持无需重新编译内核即可灵活选择分配策略；配套的测试框架覆盖基础性能、碎片率、稳定性、时间维度及内核兼容性五大测试场景，提供量化指标对比能力，既保障了分配器的可靠性，也成为连接内存管理理论与工程实践的实验教学载体。

在文件系统模块，项目针对 ArceOS 原有 VFS 的功能局限，提出了三大核心改进：通过扩展 VfsOps 与 VfsNodeOps trait 构建统一 I/O 抽象模型，支持更丰富的元数据管理与设备适配；完整实现 POSIX 标准中的 *at 系列系统调用与目录操作语义，兼容 GNU coreutils 等传统用户态程序；创新设计动态多后端挂载机制，支持在任意路径运行时挂载多种文件系统，并通过最长前缀匹配实现嵌套挂载的智能请求路由，大幅提升了文件系统的灵活性与扩展性。

5.2 核心价值

项目的核心价值体现在技术创新与实践意义两个层面：技术上，延续并深化了 ArceOS 的模块化设计理念，通过 Rust trait 抽象与动态分发技术，实现了核心模块的高内聚、低耦合，为操作系统内核的灵活扩展与组件复用提供了可复用的设计范式；实践上，内存管理模块的测试框架与文件系统的 POSIX 兼容性设计，使 Unfound OS 不仅具备运行真实应用的能力，还可作为操作系统课程的实验平台，有效弥合理论教学与工程实践之间的鸿沟。

5.3 现存局限

尽管项目已实现核心功能的突破，但仍存在以下可优化空间：内存管理模块中，Hybrid 分配器的策略选择逻辑可进一步优化，目前尚未实现基于系统负载与分配请求特征的自适应调整；测试框架的自动化程度有待提升，缺乏对测试结果的可视化分析与报告生成功能。文件系统模块中，尚未支持文件权限控制、硬链接 / 软链接等 POSIX 高级特性，且多种文件系统后端的性能适配与优化不足，在高并发 I/O 场景下的表现仍有提升空间。此外，内核整体的功耗管理、中断优化及更多硬件平台的适配工作尚未开展，限制了系统的部署范围。

5.4 未来展望

未来，Unfound OS 将围绕“功能完善、性能优化、生态扩展”三大方向持续演进：

在功能完善方面，内存管理模块将优化 Hybrid 分配器的自适应策略，引入基于机器学习的分配算法选择机制；文件系统模块将补充权限管理、链接机制、磁盘配额等 POSIX 完整特性，并支持日志型文件系统（如 EXT4）的崩溃恢复功能。同时，计划新增进程管理、进程间通信（IPC）、网络协议栈等核心模块，构建功能完备的操作系统内核。

在性能优化方面，将针对内存分配器的多线程竞争问题引入无锁设计，优化 SwitchingAllocator 的切换开销；对文件系统的路径解析与 I/O 操作进行性能调优，引入缓存机制减少磁盘访问延迟；借助 Rust 的异步编程特性，实现 I/O 操

作的异步化，提升系统的并发处理能力。

在生态扩展方面，将完善内核的硬件适配层，支持更多嵌入式设备与服务器硬件平台；构建更丰富的用户态工具链与应用生态，提升系统的实用性；开放多分配器框架与文件系统抽象的接口文档，鼓励社区参与组件开发与扩展，推动模块化操作系统内核的生态共建。

最终，Unfound OS 旨在成为一个兼具稳定性、灵活性与可扩展性的通用操作系统内核原型，既可为学术研究与教学提供实验平台，也为嵌入式系统、云原生等场景提供高性能、内存安全的内核解决方案。

第 6 章致谢

本项目基于开源社区的优秀成果进行开发，部分代码和模块参考或直接使用了他人开源项目。

本项目基于 ArceOS 内核进行二次开发，ArceOS 是一个由社区驱动、采用 Rust 语言编写的实验性模块化操作系统。ArceOS 受 Unikraft 等项目启发，支持多种架构和平台，具备丰富的内核功能。

主要依赖与引用的开源项目

- ArceOS
- axconfig-gen (内核配置工具)
- arceos-crates
- musl (C 运行时工具链)
- 其他依赖详见 Cargo.toml

本项目遵循多重开源协议（GPL-3.0-or-later OR Apache-2.0 OR MulanPSL-2.0），具体以各模块 LICENSE 文件为准。如需商用或二次分发，请务必遵守相关开源协议要求，并保留原作者署名与许可证声明。

感谢杭州电子科技大学杨浩老师和刘真老师的指导！